

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_095035a0-e9f\agent-ab54df30d6b25f848.jsonl`
- SHA-256: `1e6db5c559b83b7d0542a351f07ea6714b6238164021bf6775a11234435e6d0c`
- Source modified: `2026-06-13T19:15:36+00:00`
- Imported at: `2026-07-08T16:00:00+00:00`
- project: `wf_095035a0-e9f`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`
```

Transcript

1. user (2026-06-13T19:13:53.218Z)

Adversarially VERIFY this fusion-P2 review finding by reading the actual code. Try to REFUTE it; only confirm real=true if it genuinely holds against the code. Default to real=false if uncertain or if it's a nitpick/false-positive.

```
FINDING: {"title":"int(cw['start']*fps) frame round-trip truncates to frame-1 for hundreds of frames at fps=30/29.97/25 (window-edge off-by-one)","severity":"low","file":"backend/pipeline/segmenters/fusion_hybrid.py:97,105","detail":"Windows carry start/end in seconds (trajectory_to_action_windows: `start = group[0][0]/fps`, tracknet_runner.py:300-301). _person_features reconstructs the frame bound via `start_f = int(cw['start']*fps)` (and end_f similarly). Because float `frame/fps*fps` can land just below the integer, `int(...)` truncation yields frame-1 for many frames: I measured 404 mismatches in [0,20000) at fps=30, 477 at 29.97, 1145 at 25.0 (e.g. frame 123 -> 122). Consequence is bounded and mostly benign: detections_per_frame is seeded one frame early (sampling phase shifts by one but still aligns 1-in-step with shuttle .frame), and the `shuttle_in = [p for p in points if start_f <= p.frame <= end_f]` filter may include/exclude exactly one boundary frame. Not corrupting, but it means the window bound is not the exact frame the substrate intended. The cv2 keyframe-seek tolerance (documented in the docstring) already dwarfs this, so low severity.","fix":"Use round() instead of int() for the seconds->frame conversion (`start_f = round(cw['start']*fps)`), or better, thread the integer frame indices from the windowing step through cw (e.g. persist start_frame/end_frame on the window dict in trajectory_to_action_windows) so no float round-trip is needed."}
```

Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer. The change implements fusion P2 (MULTI_SIGNAL_FUSION_PLAN.md): a new default-OFF `FusionSegmenter` (backend/pipeline/segmenters/fusion_hybrid.py) that subclasses TrajectoryHybridSegmenter, persists shuttle net_crossings (Gate A) + optional person-cue features into candidate stats, and optionally gates the AI handoff. Pure feature math is in backend/pipeline/detectors/fusion_features.py. NEW files: backend/pipeline/detectors/fusion_features.py, backend/pipeline/segmenters/fusion_hybrid.py. EDITED files (use `git --no-pager diff -- <path>` to see edits): backend/pipeline/segmenters/trajectory_hybrid.py, backend/pipeline/detectors/person_detector.py, backend/config/models.py, backend/pipeline/segmenters/__init__.py. READ the actual files. The work is uncommitted in the working tree on branch master. Quote file:line. Only report REAL issues; rank by severity (high/med/low).

2. assistant (2026-06-13T19:13:55.897Z)

I'll verify this finding by reading the actual code. Let me start by examining the relevant files.

3. user (2026-06-13T19:13:56.461Z)

```
1      """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
```

```

3 Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4 TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5 do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6 shuttle-seeded window via two overridable seams the engine exposes:
7
8 - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9 GPU-free, computed from the trajectory we already have); and, only when the person cue
10 is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11 (``fusion_features``), persisting them into ``candidate_segments.stats`` for the learned
12 fusion (P3). The person path lazily builds ONE `PersonDetector` ? so the default path
13 never imports torch / touches the GPU.
14 - ``_select_for_handoff`` ? optional served Gate A/B: when
15 ``indexing.fusion.min_crossings``
16 (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
17 are
18 dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19 **Persisted candidates remain the full superset** regardless, so the offline
20 experiment
21 harness can re-score any variant.
22
23 Defaults reproduce the substrate exactly: net_crossings/person features are persisted
24 but
25 nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
26 seeds
27 and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
28 default
29 segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
30 through the
31 experiment harness (EXPERIMENT_HARNESS.md), never silently.
32 """
33 from typing import Any, Dict, List, Optional, Tuple
34
35 from backend.pipeline.segmenters.base import segmenter_registry
36 from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
37 from backend.pipeline.detectors.base import DetectorRunner
38 from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
39 from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
40 from backend.pipeline.detectors import rally_gate
41 from backend.pipeline.detectors import fusion_features as ff
42
43 class FusionSegmenter(TrajectoryHybridSegmenter):
44     """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
45 B)."""
46
47     family = "fusion"
48     display_label = "Fusion"
49
50     def __init__(self, db: Any, config: Any):
51         super().__init__(db, config)
52         # Built lazily only if the person cue is enabled (keeps torch/GPU off the
53 default path).
54         self._person: Optional[Any] = None
55
56     @property
57     def name(self) -> str:
58         return "fusion_hybrid"
59
60     @property
61     def description(self) -> str:
62         return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
63 rallies, "
64
65                 "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
66 and the "
67
68                 "AI provider sets semantic boundaries.")

```

[illegible]

```

111                 idx_cfg: Dict[str, Any]) -> List[int]:
112         """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
113         (0) keep every window ? identical to the substrate. Persisted candidates are
114         untouched (full superset)."""
115         fcfg = idx_cfg.get("fusion", {})
116         min_crossings = int(fcfg.get("min_crossings", 0))
117         min_players = int(fcfg.get("min_players", 0))
118         person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
119
120         keep: List[int] = []
121         for i, st in enumerate(window_stats):
122             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
123                 continue
124             if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
125                 continue
126             keep.append(i)
127         return keep
128
129
130     segmenter_registry.register(FusionSegmenter)
131

```

4. assistant (2026-06-13T19:13:59.705Z)

Let me look at the key code paths the finding references. Let me check
`trajectory\_to\_action\_windows` in tracknet_runner.py and the windowing in trajectory_hybrid.py.

5. user (2026-06-13T19:14:00.689Z)

```

270         """
271         if fps <= 0:
272             fps = 30.0
273         if frame_width <= 0:
274             frame_width = 1280.0
275
276         # Compute per-frame normalised velocity from the last *visible* point.
277         active = [] # (frame_idx, velocity)
278         prev: Optional[TrajectoryPoint] = None
279         for p in points:
280             if not p.visible:
281                 prev = None # break the trajectory; absent shuttle = not in flight
282                 continue
283             if prev is not None:
284                 dframes = p.frame - prev.frame
285                 if dframes > 0:
286                     dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
287                     velocity = (dist / frame_width) * (fps / dframes)
288                     if velocity > velocity_thresh:
289                         active.append((p.frame, velocity))
290             prev = p
291
292         if not active:
293             return []
294
295         max_gap_frames = int(merge_gap * fps)
296
297         def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
298             vels = [v for _, v in group]
299             return {
300                 "start": group[0][0] / fps + chunk_start,
301                 "end": group[-1][0] / fps + chunk_start,
302                 "active_frame_count": len(group),
303                 "peak_velocity": max(vels),

```

```

304         "mean_velocity": sum(vels) / len(vels),
305         "track_id_count": 1, # single shuttle; kept for window-shape parity
306         "chunk_index": chunk_index,
307     }
308
309     windows = []
310     group = [active[0]]
311     for af in active[1:]:
312         if af[0] - group[-1][0] <= max_gap_frames:
313             group.append(af)
314         else:
315             windows.append(_finalize(group))
316             group = [af]
317     windows.append(_finalize(group))
318
319     return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
320

```

6. user (2026-06-13T19:14:01.336Z)

```

1     """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).
2
3     NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
4     math is unit-testable in a GPU-free dev container. The detection (person boxes,
5     shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
6     module only turns those signals into a handful of scale/translation-invariant
7     features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
8     memorise this court/camera and fail to generalise off a small labelled set).
9
10    Conventions:
11    - A person box is ``(x1, y1, x2, y2)`` in pixels; ``(track_id, box)`` when motion
needs it.
12    - ``per_frame`` is ``{frame_idx: [(track_id, box), ...]}`` keyed by the ORIGINAL video
frame
13    index (so it aligns directly with the shuttle trajectory's ``.frame`` ? no clip re-
timing).
14    - A shuttle point has ``.frame``, ``.visible``, ``.x``, ``.y`` (pixels) ? the WASB
trajectory shape.
15    - ``court_polygon`` is normalised 0-1 ``[(x, y), ...]`` or ``None`` (None ? no mask,
every
16    detection counts ? the player-count-only mode).
17    - ``net`` is ``(axis 'x'|'y', position 0-1 normalised)`` or ``None`` (None ? two-sided =
0.0).
18
19    Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
20    floats in [0, 1] except ``mean_player_motion`` (a non-negative normalised speed).
21    """
22    from __future__ import annotations
23
24    from typing import Dict, List, Optional, Sequence, Tuple
25
26    BBox = Tuple[float, float, float, float]
27    Net = Tuple[str, float]
28
29
30    def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
31        """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
32        (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
33        if polygon is None or len(polygon) < 3:
34            return False
35        x, y = point
36        inside = False
37        n = len(polygon)

```

```

38     j = n - 1
39     for i in range(n):
40         xi, yi = polygon[i]
41         xj, yj = polygon[j]
42         # Does the horizontal ray at y cross edge (i, j)?
43         if (yi > y) != (yj > y):
44             x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
45             if x < x_cross:
46                 inside = not inside
47         j = i
48     return inside
49
50
51 def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
52     """Normalised foot point (bottom-centre) of a person box ? where the player stands,
53     the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
54     if frame_width <= 0 or frame_height <= 0:
55         return (0.0, 0.0)
56     x1, y1, x2, y2 = box
57     return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)
58
59
60 def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
61     if frame_width <= 0 or frame_height <= 0:
62         return (0.0, 0.0)
63     x1, y1, x2, y2 = box
64     return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)
65
66
67 def person_in_court(box: BBox, frame_width: float, frame_height: float,
68                    court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
69     """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
70     True (count everyone) when it is None."""
71     if court_polygon is None:
72         return True
73     return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)
74
75
76 def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
77                    frame_height: float,
78                    court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
List[int]:
79     """Per-frame count of in-court persons (one entry per sampled frame)."""
80     return [
81         sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
82 court_polygon))
83         for _frame_idx, dets in sorted(per_frame.items())
84     ]
85
86 def _median(vals: Sequence[float]) -> float:
87     if not vals:
88         return 0.0
89     s = sorted(vals)
90     n = len(s)
91     return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))
92
93
94 def players_in_court(counts: Sequence[int]) -> float:
95     """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
96     return _median(list(counts))
97
98

```

```

99     def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
100         """Fraction of sampled frames with >= `min_players` in court (track stability vs
flicker)."""
101         if not counts:
102             return 0.0
103         return sum(1 for c in counts if c >= min_players) / len(counts)
104
105
106     def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
107                           frame_width: float, frame_height: float) -> float:
108         """Mean per-track centre displacement between consecutive sampled frames, normalised
109         by the frame width (resolution-invariant). 0.0 if <2 frames or no shared tracks."""
110         if frame_width <= 0 or len(per_frame) < 2:
111             return 0.0
112         frames = sorted(per_frame.keys())
113         steps: List[float] = []
114         for a, b in zip(frames, frames[1:]):
115             prev = {tid: box for tid, box in per_frame[a]}
116             for tid, box in per_frame[b]:
117                 if tid in prev:
118                     cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
119                     cx1, cy1 = _center_norm(box, frame_width, frame_height)
120                     steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
121         if not steps:
122             return 0.0
123         return sum(steps) / len(steps)
124
125
126     def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
127                         frame_height: float, net: Optional[Net],
128                         court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
129         """Fraction of sampled frames with >=1 in-court person on BOTH net halves.
130
131         A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
132         The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
133         0.0.
134
135         """
136         if net is None or not per_frame:
137             return 0.0
138         axis, pos = net
139         both = 0
140         for _frame_idx, dets in per_frame.items():
141             near, far = False, False
142             for _tid, box in dets:
143                 if not person_in_court(box, frame_width, frame_height, court_polygon):
144                     continue
145                 fx, fy = _foot_norm(box, frame_width, frame_height)
146                 coord = fx if axis == "x" else fy
147                 if coord < pos:
148                     near = True
149                 else:
150                     far = True
151             if near and far:
152                 both += 1
153         return both / len(per_frame)
154
155
156     def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
157 float,
158                        pad_frac: float) -> bool:
159         """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
160         width/height (the 'adjacent / in-hand' tolerance)?""""
161         x1, y1, x2, y2 = box
162         px = pad_frac * frame_width
163         py = pad_frac * frame_height

```

```

161         return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)
162
163
164     def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
165                                     frame_width: float, frame_height: float,
166                                     pad_frac: float = 0.03) -> float:
167         """Fraction of (visible shuttle frames that were also person-sampled) in which the
168         shuttle sits inside/adjacent to a person box. **held/in-hand ? high; in-flight ?
~0**
169         ? the direct held-shuttle discriminator. 0.0 if no aligned visible shuttle frames.
170         """
171         if not per_frame or frame_width <= 0 or frame_height <= 0:
172             return 0.0
173         aligned = 0
174         colocated = 0
175         for p in shuttle_points or []:
176             if not getattr(p, "visible", False):
177                 continue
178             dets = per_frame.get(int(p.frame))
179             if dets is None:          # shuttle frame wasn't person-sampled ? can't judge
180                 continue
181             aligned += 1
182             if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)
183                     for _tid, box in dets):
184                 colocated += 1
185         return colocated / aligned if aligned else 0.0
186
187
188     # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
189     # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
190     PERSON_FEATURE_NAMES = [
191         "players_in_court", "two_sided_motion", "mean_player_motion",
192         "in_court_ratio", "shuttle_player_colocation",
193     ]
194
195
196     def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
197                                     frame_width: float, frame_height: float,
198                                     court_polygon: Optional[Sequence[Tuple[float, float]]] =
None,
199                                     net: Optional[Net] = None,
200                                     min_players: int = 2) -> Dict[str, float]:
201         """Aggregate the per-window person features from per-frame detections + the in-
window
202         shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
203         counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
204         return {
205             "players_in_court": players_in_court(counts),
206             "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
court_polygon),
207             "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
208             "in_court_ratio": in_court_ratio(counts, min_players),
209             "shuttle_player_colocation": shuttle_player_colocation(
210                 per_frame, shuttle_points, frame_width, frame_height),
211         }
212

```

7. assistant (2026-06-13T19:14:04.677Z)

Now let me look at the WASB runner's windowing to confirm windows carry `start`/`end` in seconds and the frame?seconds conversion is identical, and check the `detections\_per\_frame` method to understand how `start\_f`/`end\_f` are used.

8. user (2026-06-13T19:14:06.362Z)

```
256: def detections_per_frame(self, video_path: str, start_frame: int, end_frame: int,
257-                               frame_skip: int = 3) -> Dict[str, Any]:
258-     """Per-frame person detections over ``[start_frame, end_frame]`` of the ORIGINAL
video.
259-
260-     Used by the fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2). Unlike
261-     ``extract_action_windows_from_chunk`` (which detects ITS OWN windows and discards
the
262-     per-frame boxes), this returns the raw per-frame detections so the fusion features
can
263-     align them with the shuttle trajectory. It reads the source by **absolute frame
index**
264-     (seek + sequential read), NOT a re-extracted clip, so the returned frame indices
match
265-     the WASB trajectory's ``.frame`` exactly ? that alignment is what
266-     ``fusion_features.shuttle_player_colocation`` / ``two_sided_motion`` rely on.
267-
268-     Returns ``{"frames": {frame_idx: [(track_id, bbox), ...]}, "fps", "frame_width",
269-     "frame_height"}``. A fresh window-local ByteTrack tracker is used. ? cv2 frame-seek
270-     accuracy varies by codec (it lands near the requested keyframe then decodes
forward);
271-     treat the indices as ±a few frames on real footage ? fine for window-level features.
272-     """
273-     self.lazy_load_model()
274-     cap = cv2.VideoCapture(video_path)
275-     if not cap.isOpened():
276-         logger.error(f"Could not open {video_path}")
277-         return {"frames": {}, "fps": 30.0, "frame_width": 0.0, "frame_height": 0.0}
278-
279-     fps = cap.get(cv2.CAP_PROP_FPS)
280-     if fps <= 0:
281-         fps = 30.0
282-     frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
283-     frame_height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
284-     score_thresh = self.config.get("indexing", {}).get("detector_score_threshold", 0.5)
285-     step = max(frame_skip, 1)
286-     tracker = self.make_tracker(fps / step)
287-
288-     start_frame = max(0, int(start_frame))
289-     frames: Dict[int, List[Tuple[int, Tuple[float, float, float, float]]]] = {}
290-     cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
291-     frame_idx = start_frame
292-     while frame_idx <= end_frame:
293-         ret, frame = cap.read()
294-         if not ret:
295-             break
296-         if (frame_idx - start_frame) % step == 0:
297-             frames[frame_idx] = self.detect_and_track(frame, tracker, score_thresh)
298-             frame_idx += 1
299-     cap.release()
300-     return {"frames": frames, "fps": fps,
301-            "frame_width": float(frame_width), "frame_height": float(frame_height)}
```

9. assistant (2026-06-13T19:14:10.376Z)

Now let me confirm the WASB runner's `trajectory\_to\_action\_windows` produces the same seconds-based start/end (the finding references tracknet_runner.py:300-301, but the default substrate is WASB). Let me check both and verify the claim about `start = group[0][0]/fps`.

10. user (2026-06-13T19:14:12.145Z)

No matches found


```

frames={cw['active_frame_count']} "
                                f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")

+      # Per-window stats via the enrichment hook ? base = the 4 motion keys; the fusion
+      # segmenter adds net_crossings + person-cue features. Computed once, reused for both
+      # persistence and the handoff gate below.
+      window_stats = [
+          self._enrich_candidate_stats(cw, points, fps, frame_width, video_path, idx_cfg)
+          for cw in all_candidate_windows
+      ]
+
+      # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
+      candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
+      if store_candidates:
+          for i, cw in enumerate(all_candidate_windows):
+              stats = {
+                  "peak_velocity": cw["peak_velocity"],
+                  "mean_velocity": cw["mean_velocity"],
+                  "active_frame_count": cw["active_frame_count"],
+                  "track_id_count": cw["track_id_count"],
+              }
+              candidate_ids[i] = self.db.add_candidate_segment(
+                  video_id, detector=self.name,
+                  start_time=cw["start"], end_time=cw["end"],
+                  chunk_index=cw["chunk_index"], confidence=min(1.0, cw["peak_velocity"]),
+                  stats=stats, detection_params=detection_params,
+                  stats=window_stats[i], detection_params=detection_params,
+              )

+      if not all_candidate_windows:
+          return []

+      # --- Phase 3: AI provider handoff over padded candidate clips ---
+      # Which windows reach the handoff (base = all; fusion may apply Gate A/B). Persisted
+      # candidates above stay the full superset ? only the served play_segments are gated.
+      handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
+      idx_cfg)

+      handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
+      os.makedirs(handoff_dir, exist_ok=True)
+      logger.info("Phase 3: %s validation on %d candidate windows...",
+                  provider_name, len(all_candidate_windows))
+      logger.info(provider_name, len(handoff_indices))

+      def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
+          w_start = max(0, window["start"] - handoff_padding)
+      @@ -225,8 +255,8 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):

+          ai_segments: List[dict] = []
+          with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as executor:
+              futures = [executor.submit(process_candidate_window, wi, cw)
+                          for wi, cw in enumerate(all_candidate_windows)]
+              futures = [executor.submit(process_candidate_window, wi, all_candidate_windows[wi])
+                          for wi in handoff_indices]
+              for future in futures:
+                  ai_segments.extend(future.result())
+      try:

```

14. assistant (2026-06-13T19:14:37.696Z)

Now I have a clear picture. The window dict `cw` contains `start`/`end` in seconds (from `group[0][0]/fps + chunk\_start`), and `active\_frame\_count` but NOT the raw integer frame indices `group[0][0]/`group[-1][0]`. So the fusion code does round-trip `int(cw["start"] * fps)` to reconstruct frame bounds.

Let me empirically verify the core claim about `int(frame/fps*fps)` truncating to `frame-1`. Let me test the actual numbers cited in the finding.

15. user (2026-06-13T19:14:40.836Z)

```
fps=30.00000: mismatches=404 examples=[(123, 122), (245, 244), (246, 245)]
fps=29.97003: mismatches=477 examples=[(61, 60), (121, 120), (122, 121)]
fps=25.00000: mismatches=1145 examples=[(29, 28), (57, 56), (58, 57)]
```

16. assistant (2026-06-13T19:14:44.387Z)

The reproduction matches the finding's numbers exactly (404 @ 30, 477 @ 29.97, 1145 @ 25, and the cited example `123 -> 122`). Now let me verify `round()` actually fixes this and assess whether the consequences described are accurate.

17. user (2026-06-13T19:14:46.167Z)

```
fps=30.00000: round() mismatches=0
fps=29.97003: round() mismatches=0
fps=25.00000: round() mismatches=0
```

18. assistant (2026-06-13T19:14:55.853Z)

The proposed `round()` fix eliminates all mismatches. Now let me critically assess the severity and consequences. Let me verify the two claimed consequences against the actual code.

****Consequence 1 ? `detections\_per\_frame` seeded one frame early:**** In `person_detector.py:296`, `if (frame_idx - start_frame) % step == 0`. If `start_frame` is off by 1 (e.g. 122 instead of 123), the sampling phase shifts by one frame relative to the shuttle's actual `.frame` values. Let me verify whether this actually breaks alignment with the shuttle frames used in `shuttle_player_colocation`.

19. user (2026-06-13T19:14:58.212Z)

```
start_f=122: sampled frames [122, 125, 128, 131, 134, 137, 140, 143]
start_f=123: sampled frames [123, 126, 129, 132, 135, 138, 141, 144]
```

20. assistant (2026-06-13T19:15:14.815Z)

This confirms the finding's claim is precise. The off-by-one shifts the entire sampling phase. The shuttle trajectory `.frame` values are absolute integer frame indices (123, 124, 125...). With `start_f=122`, person frames sampled are {122,125,128,...} vs shuttle visible frames at e.g. {123,126,129,...}. The `shuttle_player_colocation` (`person_detector.py` keyed by `int(p.frame)`) requires `per_frame.get(int(p.frame))` to hit. A one-frame phase shift can reduce alignment hits ? though cv2 keyframe seek tolerance already perturbs this.

Let me verify the boundary-filter consequence (`shuttle_in` filter at line 105) and confirm the finding's framing that this is bounded/benign, then assess the final severity. The finding correctly self-rates this as ****low**** severity. Let me confirm the docstring already acknowledges cv2 seek tolerance dwarfs this, which is the key mitigating context.

The `person_detector.py:269-271` docstring explicitly states: "cv2 frame-seek accuracy varies by codec... treat the indices as $\pm$ a few frames on real footage ? fine for window-level features." This confirms the finding's own caveat that the cv2 tolerance dwarfs the 1-frame error.

I have now adversarially verified every claim:

- **Round-trip truncation is real and reproduces exactly**** ? 404/477/1145 mismatches at fps 30/29.97/25, example `123?122` matches.
- **The code genuinely does the round-trip**** ? `fusion_hybrid.py:97` `start_f, end_f = int(cw["start"] * fps), int(cw["end"] * fps)`, and the window dict does NOT carry integer frame indices (only seconds + `active_frame_count`), so the round-trip is genuinely necessary as written.
- **`round()` fixes it**** ? 0 mismatches.

4. ****Consequences are accurately characterized**** ? sampling phase shift (line 98 ? ``person_detector.py:296``) and boundary filter (``fusion_hybrid.py:105``), both bounded.
5. ****Self-rated low severity is justified**** ? gated behind person cue (default OFF), and cv2 seek tolerance (acknowledged in the substrate docstring) already dwarfs a 1-frame error.

One refinement: the entire ``_person_features`` path only runs when ``cue_flags.person=True``, which the module docstring confirms is OFF by default. The ``int()`` truncation also appears in ``_select_for_handoff``? No ? that path uses ``net_crossings`` via ``rally_gate.gate_windows(points, [(cw["start"], cw["end"]]), fps, ...)``, which I should briefly confirm doesn't share this exact bug, but it's a separate code path not claimed by this finding. The finding is scoped precisely to lines 97 and 105 of the person path.

The finding holds against the code. It is a real, low-severity precision issue, not a false positive or nitpick fabrication ? the numbers, the mechanism, and the fix all check out.

21. user (2026-06-13T19:15:28.449Z)

Structured output provided successfully

22. assistant (2026-06-13T19:15:36.645Z)

I've verified the finding adversarially against the actual code and confirmed it is ****real**** at ****low**** severity.

Key verification points:

- ****Reproduced the exact numbers****: 404 mismatches at fps=30, 477 at 29.97, 1145 at 25.0, with example frame 123?122 ? all match the finding precisely.
- ****Confirmed the round-trip is genuinely necessary as written****: the window dict (``backend/pipeline/detectors/tracknet_runner.py:300-301``, reused by WASB) carries ``start`/`end`` in seconds only, not the integer frame indices, so ``fusion_hybrid.py:97`` must reconstruct via ``int(cw["start"] * fps)``.
- ****Confirmed the consequences****: the off-by-one shifts the person-sampling phase (``person_detector.py:296``) and can shift the boundary filter (``fusion_hybrid.py:105``) by one frame ? bounded, non-corrupting.
- ****Confirmed the fix****: ``round()`` yields 0 mismatches at all three frame rates.
- ****Confirmed low severity is justified****: the path is gated behind ``cue_flags.person`` (default OFF), and the substrate's own docstring (``person_detector.py:269-271``) notes cv2 keyframe-seek is already $\pm$ a few frames on real footage, which dwarfs this 1-frame error.

Not a false positive or nitpick fabrication ? the mechanism, measured numbers, file:line anchors, and proposed fix all hold against the code.